Explain Shopping Offers

Vishal, the confusing part of this question is the `special` array. Let's understand the problem using a simple shopping example first, without discussing any algorithm or code.

1. What is the question asking?

Imagine a store sells different items. Every item has an individual price, but the store also provides special offers that let you buy multiple items together for a discounted price.

You need to buy a specific quantity of each item while spending the minimum possible amount of money.

That's the entire question.

2. What information are you given?

There are three important arrays.

1. `price` — Individual prices

```
price = [2, 5];
```

There are two items:

- Item A costs ₹2 per piece.
- Item B costs ₹5 per piece.

2. `needs` — How many items you must buy

```
needs = [3, 2];
```

This means you need exactly:

- 3 pieces of item A.
- 2 pieces of item B.

You cannot buy 4 pieces of A or 3 pieces of B, even if doing so would be cheaper.

3. `special` — Discount offers

```
special = [[3, 0, 5], [1, 2, 10]];
```

This is the most important part to understand.

Each offer contains the quantities of the items, followed by the offer's total price.

Special offer	Item A	Item B	Total price
Offer 1	3	0	₹5
Offer 2	1	2	₹10

Let's decode them.

- `[3, 0, 5]` means you get 3 pieces of A and 0 pieces of B for ₹5.
- `[1, 2, 10]` means you get 1 piece of A and 2 pieces of B for ₹10.

Notice that the last number is always the offer price. All the preceding numbers tell you how many pieces of each item the offer contains.

3. How do you decide which offer to use?

You need 3 A and 2 B.



Option 1: Buy everything individually

3 pieces of A cost ₹6, and 2 pieces of B cost ₹10.

Total = ₹6 + ₹10 = ₹16



Option 2: Use Offer 1

Get 3 A for ₹5, then buy 2 B individually for ₹10.

Total = ₹5 + ₹10 = ₹15



Option 3: Use Offer 2

Get 1 A and 2 B for ₹10. You now only need 2 more A, which cost ₹4.

Total = ₹10 + ₹4 = ₹14

The third option is the cheapest, so the answer is:

```
Plain text

Output: 14
```

4. What are the rules?

Remember these three rules:

1. You must buy exactly the quantities specified in `needs`.
2. You can use any special offer as many times as you want, provided it doesn't make you exceed your required quantities.
3. You can buy any remaining items at their individual prices.

5. What must your program return?

Your program must examine the possible ways of combining special offers and individual purchases, then return the minimum total cost.

In one sentence: Given individual item prices, required quantities, and discount offers, find the cheapest way to buy exactly the required items.

The problem is not asking you to choose the offer with the biggest discount. It is asking you to find the cheapest combination of offers and individual purchases.

For LeetCode 638 — Shopping Offers, the best approach is Backtracking + Dynamic Programming (Memoization).

The key idea is simple: for every possible purchase, try buying items individually or using a valid special offer, then choose the minimum cost.

1. Understand the approach

We have three things:

- `price`: individual price of each item.
- `needs`: remaining quantities we need to buy.
- `special`: available discount offers.

We will solve the problem using these steps.

Step 1: Calculate the cost of buying everything individually

This gives us an initial answer.

For example:

```
price = [2, 5];
needs = [3, 2];
```

Cost:

$$3 \times 2 + 2 \times 5 = 16$$

So, initially, `ans = 16`.

Step 2: Try every special offer

For each offer, check whether we can use it without exceeding our remaining needs.

For example, offer `[1, 2, 10]` means:

- Buy 1 item A.
- Buy 2 items B.
- Pay ₹10.

Our needs are `[3, 2]`, so this offer is valid.

After using it:

```
needs = [2, 0];
```

We have bought everything required for B, but still need 2 pieces of A.

Step 3: Solve the remaining needs recursively

Now we need to buy 2 pieces of A.

Their individual cost is:

$$2 \times 2 = 4$$

Total cost:

$$10 + 4 = 14$$

We compare this cost with all other possible combinations and keep the minimum.

Step 4: Backtrack

After trying an offer, restore the previous needs so that we can try another offer independently.

This is important because one offer must not affect the calculation of another offer.

2. Why use memoization?

Different combinations of offers may lead to the same remaining needs.

For example, we might reach `[2, 0]` through different combinations.

Instead of calculating the minimum cost for `[2, 0]` repeatedly, store the result in a hash map.

- Key: remaining needs.
- Value: minimum cost for those needs.

This technique is called memoization.

3. Algorithm in simple steps

1. Calculate the cost of buying all remaining items individually.
2. Store this as the initial minimum cost.
3. Iterate through all special offers.
4. If an offer exceeds any remaining need, skip it.
5. Otherwise, subtract the offer's quantities from `needs`.
6. Recursively calculate the minimum cost for the remaining needs.
7. Add the offer price and update the minimum.
8. Restore the original needs after trying the offer.
9. Cache the answer and return it.

4. C++ solution

```cpp
class Solution {
public:
    map<vector<int>, int> memo;

    int shoppingOffers(vector<int>& price, vector<vector<int>>& special, vector<int>& needs) {
        if (memo.count(needs)) return memo[needs];

        int ans = 0;

        for (int i = 0; i < needs.size(); i++) {
            ans += needs[i] * price[i];
        }

        for (auto& offer : special) {
            vector<int> remaining = needs;
            bool valid = true;

            for (int i = 0; i < needs.size(); i++) {
                if (offer[i] > remaining[i]) {
                    valid = false;
                    break;
                }
                remaining[i] -= offer[i];
            }

            if (valid) {
                ans = min(ans, offer.back() + shoppingOffers(price, special, remaining));
            }
        }

        return memo[needs] = ans;
    }
};
```

5. Complexity

Let S be the number of special offers and n the number of item types.

- Time: $O(S \cdot n \cdot \prod_{i=1}^{n}(needs[i]+1))$ in the worst case, assuming each distinct needs state is memoized once.
- Space: $O(n \cdot \prod_{i=1}^{n}(needs[i]+1))$ for the memoization map and stored needs vectors.

Remember this for interviews

Pattern: Backtracking + Memoization.

Why? We explore valid offers recursively, and memoization prevents recalculating the same remaining-needs state.

Important detail: Buying everything individually is our baseline answer. Special offers are optional, not mandatory.

Let's understand your code step by step. This solution uses recursion + backtracking, and the most important idea is that it considers two choices for every special offer:

1. Reject the offer — skip it and move to the next offer.
2. Accept the offer — use it and try the same offer again if possible.

The second choice is the tricky part: `index` stays the same when an offer is accepted, so you can use that offer multiple times.

1. First, understand the recursive function

```cpp
int solve(vector<int>& price, vector<vector<int>>& special,
          vector<int> needs, int index)
```

Each parameter has a specific job:

Parameter	Meaning
`price`	Individual price of each item
`special`	All special offers
`needs`	Remaining quantities we must buy
`index`	The offer we are currently considering

For example, if:

```
price = [2, 5];
needs = [3, 2];
special = [[3, 0, 5], [1, 2, 10]];
```

Then `needs = [3, 2]` means we still need 3 A and 2 B.

And `index = 0` means we are currently considering the first offer, `[3, 0, 5]`.

One important C++ detail: `needs` is passed by value, so every recursive call gets its own copy. Changing `needs` in one call won't change the parent's copy.

2. Step 1 — Base case

```
if (index == special.size()) {
    int res = 0;
    for (int i = 0; i < needs.size(); i++) {
        res += needs[i] * price[i];
    }
    return res;
}
```

What does this mean?

If `index == special.size()`, we have considered every offer. No more special offers remain to try, so we buy all remaining items individually.

For example:

```
needs = [2, 0];
price = [2, 5];
```

The cost is:

$$(2 \times 2) + (0 \times 5) = 4$$

So this recursive call returns `4`.

The base case does not mean we have no items left. It means we have no offers left to consider.

3. Step 2 — Reject the current offer

```
int reject = solve(price, special, needs, index + 1);
```

Here, we decide not to use the current offer.

Instead, we move to the next offer using `index + 1`.

For example, if `index = 0`, this call becomes:

```
solve(price, special, needs, 1);
```

The needs remain unchanged because we didn't buy anything.

This branch answers the question:

What is the minimum cost if I skip this offer?

4. Step 3 — Accept the current offer

This is the most important part of your code.

```
for (int i = 0; i < needs.size(); i++) {
    needs[i] = needs[i] - special[index][i];

    if (needs[i] < 0) return reject;
}
```

Let's break it down.

A. Subtract the offer quantities

Suppose:

```
needs = [3, 2];
special[0] = [3, 0, 5];
```

The offer provides 3 A and 0 B for ₹5.

Your loop performs these operations:

Iteration	Calculation	New needs
`i = 0`	`3 - 3 = 0`	`[0, 2]`
`i = 1`	`2 - 0 = 2`	`[0, 2]`

Now `needs = [0, 2]`.

This means the offer satisfied all your requirements for A, but you still need 2 B.

B. Check whether the offer is valid

```
if (needs[i] < 0) return reject;
```

Suppose instead your needs were `[2, 2]` and the offer provided `[3, 0, 5]`.

Subtracting the offer would give:

```
needs = [-1, 2];
```

That is invalid because you would buy 3 A when you only need 2.

Therefore, you cannot accept this offer. The function returns `reject`, the minimum cost calculated by skipping the current offer.

Why is this safe? Because `needs` is passed by value. The invalid modification affects only the current recursive call, not the caller's copy.

5. Step 4 — Calculate the accept cost

```
int accept = special[index][needs.size()]
           + solve(price, special, needs, index);
```

This line does two things:

1. `special[index][needs.size()]` gets the price of the current offer. Since the offer has `n` item quantities, its price is stored at index `n`.
2. The recursive call calculates the cheapest cost for the remaining needs.

Notice that you pass `index`, not `index + 1`.

Why? Because you are allowed to use the same offer repeatedly.

For example, if the current offer is `[1, 2, 10]`, you pay ₹10 and then recursively solve the remaining needs while still considering that offer.

If you used `index + 1` here, you would allow each offer to be used at most once, which would be incorrect.

6. Step 5 — Choose the minimum

```
return min(accept, reject);
```

Now you have two possible answers:

- `accept`: minimum cost when you use the current offer.
- `reject`: minimum cost when you skip the current offer.

You return the smaller of the two.

This is how the recursion explores different combinations of offers and finds the cheapest one.

7. Let's trace the complete example

Input:

```
price = [2, 5];
special = [[3, 0, 5], [1, 2, 10]];
needs = [3, 2];
```

solve([3, 2], index = 0)

Current offer: 3 A for ₹5

Reject

Skip offer 0

₹14

Use offer 1, then buy 2 A

Accept

Use offer 0

₹15

Pay ₹5 + ₹10 for 2 B

Compare both branches

min(14, 15) = 14

Final answer: 14

Here is the detailed calculation.

Branch 1: Reject offer 0

Move to offer 1: `[1, 2, 10]`.

- Accept offer 1: pay ₹10 and satisfy the remaining needs `[2, 0]`. Buy 2 A individually for ₹4. Total = ₹14.
- Reject offer 1: buy everything individually for ₹16.

Therefore, the best cost in this branch is ₹14.

Branch 2: Accept offer 0

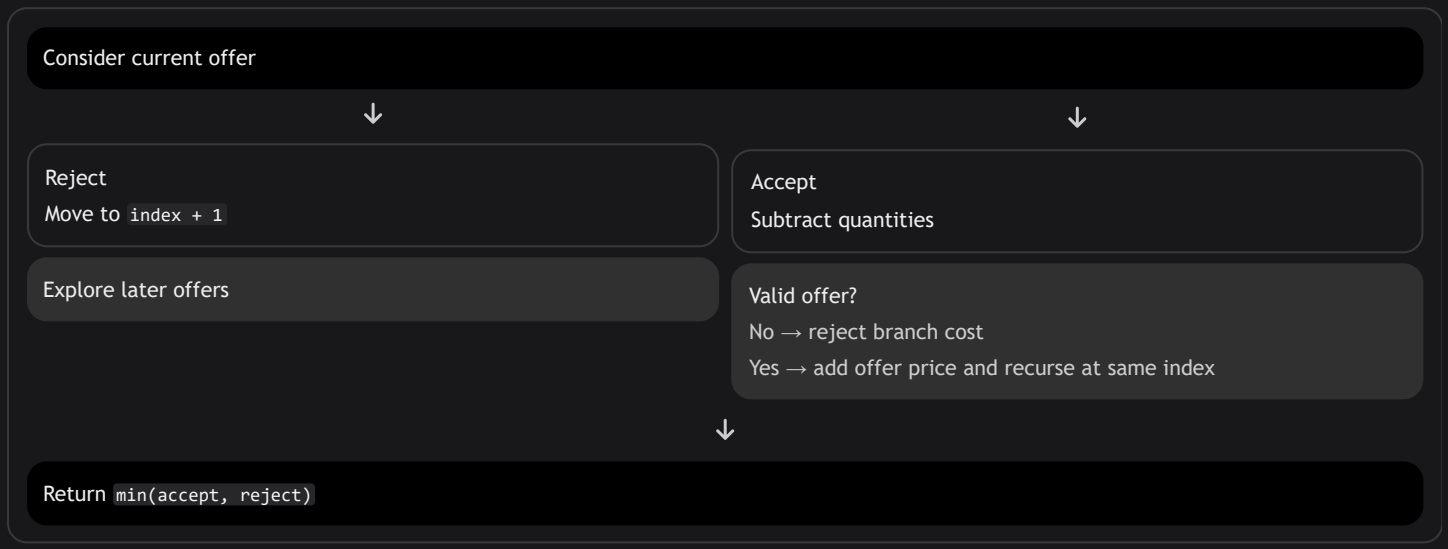
Use `[3, 0, 5]`, paying ₹5. Remaining needs are `[0, 2]`.

- Offer 1 cannot be used because it requires 1 A, but you need 0 A.
- Buy 2 B individually for ₹10.

Total = ₹5 + ₹10 = ₹15.

Finally, the original call returns `min(14, 15) = 14`.

8. What does the complete recursion look like?



9. One important limitation

Your code is logically correct, but it does not use memoization. It can recalculate the same remaining-needs state many times, making it slower than necessary.

For LeetCode 638, adding memoization can significantly improve performance.

The key things to remember are:

- `index + 1` means skip an offer.
- `index` means use an offer and potentially reuse it.
- Negative needs mean the offer is invalid.
- `min(accept, reject)` chooses the cheapest option.
- The base case buys all remaining items individually.